

Projet semestriel : Deep Learning

Marie Pierre Gnilane Ndew FAYE

M1 Mathématiques appliquées aux données
Université Sorbonne Paris Nord / Institut Galilée

Année académique 2025–2026

Encadrant : M. Charles DE CLERCQ

Tuteur académique : M. Pascal BOYER

- ① Contexte et motivation
- ② Cadre probabiliste du machine learning
- ③ Réseaux de neurones shallow
- ④ Réseaux de neurones deep
- ⑤ Apprentissage et comparaison

- Explosion des données $\Rightarrow$ besoin de modèles apprenant à partir des données
- ML : apprendre une relation $y \approx f(x; \theta)$ sans règles écrites à la main
- Deep learning : succès sur images, texte, audio grâce aux réseaux profonds
- Enjeu : comprendre **représentation** + **apprentissage** + **compromis**

- Données supervisées : $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- Modèle probabiliste : $\Pr(y \mid x; \theta)$
- Vraisemblance : $\Pr(\mathcal{D} \mid \theta) = \prod_{i=1}^N \Pr(y_i \mid x_i; \theta)$
- Apprentissage ML : $\theta^* = \arg \max_{\theta} \Pr(\mathcal{D} \mid \theta)$
- Perte : $L(\theta) = - \sum_{i=1}^N \log \Pr(y_i \mid x_i; \theta)$

Objectif du machine learning

- Hypothèse : relation inconnue $y = g(x)$
- On apprend un modèle paramétré $f(x; \theta)$
- But : bonne précision sur les données + **généralisation** sur de nouvelles données

- Entrée : $x = (x_1, \dots, x_D)^\top$
- Pré-activation : $z = \sum_{j=1}^D \theta_j x_j + \theta_0$
- Activation : $h = a(z)$ (non-linéarité indispensable)
- Exemple ReLU :

$$\text{ReLU}(z) = \begin{cases} 0 & \text{si } z < 0 \\ z & \text{si } z \geq 0 \end{cases}$$

Shallow neural network : propagation avant

- 1 couche cachée de H neurones
- Couche cachée :

$$z_i^{(1)} = \sum_{j=1}^D \theta_{ij}^{(1)} x_j + \theta_{i0}^{(1)}, \quad h_i = a(z_i^{(1)})$$

- Sortie (K neurones) :

$$y_k = \sum_{i=1}^H \theta_{ki}^{(2)} h_i + \theta_{k0}^{(2)}$$

Théorème d'approximation universelle (idée)

- Un réseau à 1 couche cachée peut approximer toute fonction continue (sur un compact)
- MAIS : résultat d'existence
 - ne dit pas combien de neurones il faut
 - ne garantit pas un apprentissage facile/rapide

- Fonctions complexes $\Rightarrow$ besoin de **beaucoup** de neurones (réseau très large)
- Coût mémoire/temps $\uparrow$
- Représentations non-hiérarchiques : “tout en une seule étape”
- Motivation : empiler des couches $\Rightarrow$ deep learning

- Plusieurs couches cachées (L)
- Composition de transformations :

$$z_i^{(\ell)} = \theta_{i0}^{(\ell)} + \sum_j \theta_{ij}^{(\ell)} h_j^{(\ell-1)}, \quad h_i^{(\ell)} = a(z_i^{(\ell)})$$

- Lecture : profondeur = nombre de compositions ; largeur = richesse par couche

- Apprentissage de représentations **hiérarchiques**
- Moins de *feature engineering* manuel
- Certaines fonctions : représentation plus **efficace** en deep qu'en shallow
- Succès sur données complexes (images, langage, audio)

Apprentissage : perte = -log-vraisemblance

- Cadre général :

$$L(\theta) = - \sum_{i=1}^N \log \Pr(y_i \mid x_i; \theta)$$

- Régression + bruit gaussien : $y_i = f(x_i; \theta) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$
- $\Rightarrow$ perte quadratique :

$$L(\theta) = \sum_{i=1}^N (y_i - f(x_i; \theta))^2$$

- Problème : $\theta^* = \arg \min_{\theta} L(\theta)$ (non convexe)

- Mise à jour :

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} L(\theta^{(t)})$$

- En réseaux : gradients calculés efficacement par **backpropagation** (règle de la chaîne)
- Sensible à : taux d'apprentissage η , initialisation, normalisation, etc.

Comparaison shallow vs deep : exemple du rapport

- Deux réseaux avec le **même nombre de paramètres** : 286
- Shallow : 1 couche cachée de largeur 95
- Deep : 10 couches de largeur 5
- 1D + ReLU : nombre de régions linéaires

$$R_{\text{shallow}} \leq 96, \quad R_{\text{deep}} \leq (5 + 1)^{10} = 60\,466\,176$$

- Message : à paramètres égaux, la profondeur peut donner une expressivité bien supérieure

- Shallow : universel en théorie, mais souvent inefficace (trop large)
- Deep : représentations hiérarchiques + forte expressivité
- Apprentissage : perte probabiliste + descente de gradient/backprop
- Compromis central : expressivité vs coût de calcul vs optimisation

Réf. principale : S.J.D. Prince, *Understanding Deep Learning*, MIT Press (2023).